

完整的分步操作手册

BugIt

VS Code 的 QA 缺陷提交 Agent

首次设置与日常使用：为从未做过类似操作的人编写的详细分步指南，逐步点击讲解。无需任何技术背景。按顺序操作，你今天就能提交规范的缺陷工单。

运行环境：VS Code。Windows（主要）、macOS 和 Linux

翻译说明。

本文档由机器翻译生成，未经母语者审校。以英文版本为准：如有出入，以英文文本为准。如需最准确、最新的表述，请参阅英文文档。

本指南内容

请按顺序完成一次**第一部分**。之后的内容你可以在需要时随时查阅。末尾的**故障排查**和**常见问题**解答了最常见的问题，请在发邮件寻求支持之前先查看这些内容。

第一部分 · 设置（只需完成一次）

步骤 0：获取 GitHub Copilot

步骤 1：安装 VS Code

步骤 2：解压你下载的 BugIt

步骤 3：在 VS Code 中打开 BugIt

步骤 4：在聊天中选择 BugIt agent

步骤 5：激活你的许可证

步骤 6：接受条款（仅需一次）

步骤 7：运行 "Begin setup"

步骤 8：连接你的 tracker

步骤 9：保存你的登录信息

步骤 10：确认你已就绪

第二部分 · 日常使用

第三部分 · 打造专属设置

第四部分 · 更新、备份与安全

第五部分 · 故障排查与常见问题

第六部分 · 许可证与支持

让你安全无虞的唯一规则

BugIt 绝不会在向你展示预览之前提交任何内容。不可逆的操作需要你输入 **FILE IT**，仅输入"是"不会执行。你始终掌控全局。想零风险练习一下？输入 **dry run**，它会写出完整的报告，但**不会**提交。

首次设置 vs. 日常使用

你只需完整走一遍**第一部分**一次。之后，打开 BugIt 会很快，更新也只需一条命令（**Update**）。详见第四部分。

第一部分 · 设置：只需完成一次

大约需要 15 分钟。请按顺序完成各步骤，每一步都建立在上一步的基础上。不要跳步。

0 获取 GitHub Copilot

没有它你就无法使用 BugIt

BugIt 是"驾驶员"，GitHub Copilot 是"引擎"（真正撰写报告的 AI）。如果你还没能让 Copilot 在 VS Code 中正常工作，下面的内容都无济于事。请先完成这一步。

它是什么：GitHub Copilot 是 GitHub（微软旗下）出品的 AI 助手。它有免费版供轻度使用，也有付费版（约每月 \$10）供重度使用。你会在步骤 1 中安装它并登录。

1. 你需要一个 **GitHub** 账户。如果还没有，可在 github.com/signup 免费注册一个。
2. Copilot 的免费版足够你试用 BugIt。如果你整天都在提交缺陷，付费的 "Copilot Pro" 套餐物有所值，你可以随时在 github.com/features/copilot 升级。
3. 你会在步骤 1 中实际在 VS Code 里开启 Copilot，这里暂时不用做任何其他事。

1 安装 VS Code

VS Code 是微软出品的免费应用。它是 BugIt 所在的"窗口"。

1. 打开浏览器，访问 code.visualstudio.com。
2. 点击蓝色的大 **Download** 按钮（它会自动识别你的系统）。
3. 打开下载的文件并安装：**接受所有默认选项**，一路点击 Next/Install 即可。
4. 安装完成后打开 VS Code。

在 VS Code 中开启 GitHub Copilot

1. 在 VS Code 最左侧，点击 **Extensions** 图标（看起来像四个小方块）。
2. 在搜索框中输入 **GitHub Copilot**。
3. 点击 "GitHub Copilot" 上的 **Install**（如果有提供，也安装 "GitHub Copilot Chat"）。
4. 系统会提示你登录 **GitHub**。点击它，然后在弹出的浏览器窗口中用你的 GitHub 账户登录。

✅ **成功的标志**：VS Code 底部会出现一个小的 Copilot 图标，聊天面板也会打开（左侧的对话气泡图标，或按 **Ctrl+Alt+I**）。

2 解压你下载的 BugIt

你购买的产品附带一个 .zip 文件（例如 [bugit-qa-agent.zip](#)）。你需要先解压它，保持压缩状态是无法运行的。

1. 找到该 .zip 文件（通常在下载文件夹中）。
2. 右键点击它 → 选择 **Extract All...**（Windows）或双击它（Mac）。
3. 选择一个简单的位置，比如你的**文档**文件夹，然后点击 Extract。
4. 现在你会得到一个包含大量文件的 **BugIt** 文件夹。记住它的位置。

不要放进 OneDrive 或其他同步文件夹

云同步文件夹可能导致文件冲突。放在"文档"或"桌面"里最合适。另外也不要重命名文件夹内的文件，保持原样即可。

3 在 VS Code 中打开 BugIt

1. 在 VS Code 中，点击 **File → Open Folder**（左上角菜单）。
2. 导航到你解压 BugIt 的位置（例如"文档"）。
3. 单击一次 **BugIt** 文件夹以选中它，然后点击 **Select Folder**。
4. 如果 VS Code 询问 "Do you trust the authors of the files in this folder?"，点击 **Yes, I trust the authors**。

✅ **成功的标志**：你会在 VS Code 左侧看到 BugIt 的文件列表（比如 [tools](#)、[.github](#) 等文件夹，以及 [README.md](#) 等文件）。

4 在聊天中选择 Buglt agent

1. 打开 Copilot **Chat** 面板：点击左侧的对话气泡图标，或按 **Ctrl+Alt+I** (Windows) / **Cmd+Alt+I** (Mac)。
2. 聊天框顶部有一个小的**模式**下拉菜单（可能显示 "Ask" 或 "Agent"）。
3. 点击它，从列表中选择 **Buglt QA Agent**。

列表中看不到 "Buglt QA Agent" ？

请确认你打开的是 Buglt 文件夹（步骤 3），而不是其他文件夹，因为 agent 是从这个文件夹内部加载的。如有需要，关闭并重新打开该文件夹。

5 激活你的许可证

你只需要一个 **Buglt** 账户，无需复制或粘贴任何内容。激活在你的浏览器中完成。

1. 在聊天框中输入 **hi** 并按回车。
2. 输入 **Activate**；Buglt 会在你的浏览器中打开 Buglt Portal；登录你自己的账户，选择你的 Solo 或 Team 权益，并批准这台设备；然后返回 VS Code，激活会自动完成。你的密码始终留在浏览器中，绝不会输入到 VS Code 里。

✅ **成功的标志**：Buglt 会回复 "✅ Activated: licensed to [your name/email]."

请对你的账户保密

你的账户和权益与你本人绑定。请勿分享、公开发布或转卖，被共享的访问权限可能被撤销。Team 成员各自使用自己的账户登录。没有共享的密钥，也没有共享的登录。

6 接受条款（仅需一次）

第一次使用时，Buglt 会展示一份简短摘要，并要求你先接受才能继续。

1. 阅读它展示的简短摘要（你会在每份工单提交前进行审阅；你项目的安全由你自己负责；这些保障措施是辅助手段，而非保证，而且它们由 Copilot 中当前回答你的 AI 模型来执行，较轻量/免费的模型可能偶尔会跳过某一步或需要你重复一条命令，而更强的模型则不会）。
2. 回复 **I agree** 并按回车。

✅ **成功的标志**：Buglt 会确认并继续进行设置。系统只会问你这一次，之后它会记住。

7 运行 "Begin setup"

现在 Buglt 会用大白话向你提问，从而完成自我配置。你永远不需要手动编辑设置文件。

YOU TYPE

Begin setup

它会问一些简短的问题，只开启你选中的功能。其中最重要的是你的 tracker：

它会问.....

如何回答

你使用哪个 tracker？ (选一个，必填)	你团队存放缺陷的地方。 Jira 和 Azure DevOps 提供引导式设置和经过测试的字段映射，severity/priority 会自动设置以匹配你的报告。Jira 使用带浏览器登录的 Atlassian Rovo MCP；Azure DevOps 使用 Microsoft 面向组织的远程 MCP（一项公开预览），同样通过浏览器登录。Sentry、GitHub、Linear 和 Notion 使用 BugIt 设置并在你的账户上实时验证的已知端点。在这些检查通过之前，请将它们视为实验性。其余所有工具（GitLab、Shortcut、YouTrack、Bugzilla、ClickUp、Asana、Trello）通过你自己的兼容 MCP server 连接，BugIt 会帮你完成设置。选择你团队已经在用的那个。
崩溃工具？ (可选)	Sentry 使用 BugIt 在你的账户上实时验证的已知端点（实验性）；Crashlytics、BugSnag、Raygun、Rollbar、App Center 和 Datadog 通过你自己的兼容 MCP server 连接，仅在你团队使用它们时才需要。
测试管理？ (可选)	TestRail、Xray、Zephyr、Qase。
把缺陷发布到聊天？ (可选)	Slack、Microsoft Teams、Discord 或电子邮件。粘贴一个 webhook 地址（邮件则填写你的邮件服务器信息），BugIt 会发送一条真实的测试消息，只有送达后才保存设置。你可以按频道选择哪些事件（已提交／已评论／已添加附件）以及最低严重程度，让发布频道只接收 Critical 和 High。发送失败绝不会导致提交失败。
规格文档/知识库？ (可选)	Confluence、Notion、SharePoint、Google Docs，或本仓库中的本地 Markdown 文件夹。
语言	你的主语言（默认英语）。如果你还想用第二语言提交，它会问一个简单的是/否问题。大多数团队只用一种语言。
强力功能？ (可选)	额外工具，例如分类摘要、撤销和离线导出，见第二部分。全部可选。

提示：从模板开始

如果你的工作符合某个类别，输入 `preset.py game`（还有 `web-saas`、`mobile`、`enterprise`）即可预填合理的类别和设置。之后随时可以更改。

设置暂停或被中断？重新说一遍就行

如果 `Begin setup` 中途停止（你关闭了 VS Code，或者它只是暂停了），再次输入 `Begin setup` 就会从中断的地方继续。它不会让你重新回答已经回答过的问题。想以后专门修改某项设置（添加 tracker、换一个选项），直接说出来即可，例如 "add a tracker"，它就会重新打开对应的部分。

8 连接你的 tracker ("bridge")

你的 tracker 通过一座小小的 bridge（其技术名称是 "MCP server"）与 BugIt 通信。BugIt 会替你完成这一设置，你只需回答问题并点击一个按钮。你永远不需要编辑文件。

8a · 让 BugIt 配置 bridge

- 在设置过程中，BugIt 会主动提出连接你的 tracker。对于 Jira 和 Confluence（Atlassian Rovo），它知道引导式连接；对于 Sentry、GitHub、Linear 和 Notion，它知道标准端点并会在你的账户上实时验证。它询问时你只需确认，它就会在你依赖它之前运行这些检查（在检查通过之前，请将它们视为实验性）。
- 对于自建工具或组织提供的工具，它会用一句话告诉你在哪里找到你的兼容 MCP server 地址，并让你把它粘贴到聊天框中，然后它会地址写入设置。

8b · 打开 bridge

- 在左侧的文件列表中，打开文件 `.vscode/mcp.json`（单击一次）。

- 2. 在文件内，找到写着 `► Start | More...` 的灰色小字，它就在每个 server 名称的正上方。字很小，容易错过。
- 3. 点击 BugIt 让你启动的每个 server 上的 **Start**（通常只是你选的那一个 tracker）。

8c · 在系统提示时登录

bridge 首次连接时，需要验证确实是你本人。会发生以下两种情况之一：

- 会弹出一个浏览器窗口 → 像平时一样登录你的 tracker（GitHub 和 Azure DevOps 常见）。
- VS Code 顶部会出现一个小方框 → 它会要求输入你的邮箱和/或访问令牌。粘贴进去并按回车。

去哪里获取访问令牌？

令牌就像 tracker 给你的一次性密码。在你 tracker 的网站上创建一个，然后在方框询问时粘贴进去。以下是几个常用工具的对应页面，用你平时使用的账户登录：

你的 tracker	在哪里创建令牌
Jira / Confluence (Atlassian)	<code>id.atlassian.com/manage-profile/security/api-tokens</code>
GitHub	<code>github.com/settings/tokens</code> （或直接通过浏览器登录）
GitLab	<code>gitlab.com/-/user_settings/personal_access_tokens</code>
Sentry	sentry.io → Settings → Account → API → Auth Tokens
Linear	linear.app → Settings → API → Personal API keys
Azure DevOps	具有 Work Items: Read & write 权限的 Personal Access Token（ <code>dev.azure.com/YOUR-ORG/_usersSettings/tokens</code> ）。

令牌一显示就立刻复制

大多数服务的新令牌只会显示一次。立刻复制它，并妥善保存，直到 BugIt 替你保存好（步骤 9）。选择你 tracker 提供的最长有效期，这样就不必很快再重来一遍。

8d · 确认已连接

启动某个 bridge 后，它的灰色文字会变为 "Running" 或显示一个绿点。为保险起见，在聊天框中输入：

```
YOU TYPE
Check status
```

✓ 成功的标志：BugIt 会把你的 tracker 列为已连接/健康状态。

关闭 VS Code 后 bridge 会关闭

每次重新打开 VS Code，都要在 `.vscode/mcp.json` 中重新点击你 server 上的 **Start**。你保存的登录信息会被记住（步骤 9），你只需重启 bridge。这是 VS Code 的行为，并非 BugIt 的问题。

9 保存你的登录信息

当你在步骤 8 中登录 bridge 时，你的凭据会被安全地存放在你电脑内置的保险库中（Windows Credential Manager / macOS Keychain / Linux Secret Service），绝不以明文文件保存，也绝不会发送到任何地方。想确认哪

些连接器已完成身份验证，输入：

YOU TYPE

Check saved credentials

.....BugIt 会显示哪些连接器拥有身份验证，但从不显示凭据值。BugIt 从不显示或复制你保存的令牌值。

10 确认你已就绪

在正式开始提交之前，让 BugIt 帮你再检查一遍：

YOU TYPE

Check readiness

它会列出仍缺少的内容（通常只是“启动你的 bridge”或“登录”）。当一切都显示绿色时，你会看到 “ Setup complete.”

设置到此完成，一劳永逸

你不会再重复第一部分。从现在起，打开 BugIt 只需：打开文件夹 → 在 `.vscode/mcp.json` 中启动你的 bridge → 输入 `hi`。之后想更改任何选项，直接输入 `Begin setup` 并说明要改什么即可。

没有 GitHub Copilot？还有终端向导

如果你无法使用 Copilot，打开内置终端（Terminal → New Terminal）并运行 `python tools/setup_wizard.py`。它会问几个问题，并在没有 AI 的情况下替你写好设置。你也可以使用自己的 AI 密钥独立运行 BugIt，见常见问题。

第二部分 · 日常使用

设置完成后，你完全可以只在聊天中工作。可以自然地描述，也可以使用下面的确切示例。随时输入 `help` 查看完整列表。

撰写缺陷报告

这是 BugIt 的主要工作。用一句大白话描述缺陷；它会撰写完整工单，自动填写版本和类别，检查重复项，展示预览，并在你输入 `FILE IT` 。

YOU TYPE

Write a bug report: 在iPhone上点击保存时应用每次都会崩溃

它会起草标题、重现步骤、预期与实际结果、严重程度和平台，并且总是把 tracker 自己的 priority/severity 字段设置为匹配值，而不只是一个通用默认值。想在提交前做些修改？直接说出来即可，例如“把严重程度改为高”或“补充说明是从最新版本开始出现的”。

快速缺陷（快速提交）

对于常见类型（崩溃、布局、卡顿、缺失、阻断性.....），快速表单会跳过一些问题，并替你填好类别和优先级，而且仍会先运行完整的重复检查。

YOU TYPE (EXAMPLES)

Quick bug: 更新后启动崩溃

Quick bug: layout 设置界面的按钮和文字重叠了

Quick bug: blocker 支付步骤卡住无法继续

Quick bug: slow 仪表盘要花 20 秒才能加载完

崩溃缺陷报告

如果你连接了崩溃工具（如 Sentry），BugIt 会完成上述所有工作，并额外把你的报告与崩溃匹配，替你提取堆栈跟踪信息。

YOU TYPE

Write a crash bug report: 打开地图时游戏崩溃

验证与关闭评论

在你测试完修复后，BugIt 会撰写一条整洁的评论供你发布到工单上。它只撰写（并可选择发布）评论，不会更改工单的状态。你仍需自己在 tracker 中关闭/解决/重新打开该工单。

你输入	你会得到
Close #123	询问是哪种场景（修复已确认、按要求关闭、按设计如此、重新打开.....），然后撰写对应的评论。
Write a verification comment for 1.2.0 on Windows	一条"修复已确认"的评论，并填好构建版本详情。
Write a reopen comment for 1.2.0 on Windows	一条"问题仍然存在"的评论（之后你自己重新打开该工单）。

翻译

使用你团队的确切措辞（来自你的术语表，见第三部分），在你已配置的语言之间翻译报告或术语。

YOU TYPE (EXAMPLES)

Translate to ja: 个人资料页面上的保存按钮没有反应

Translate this bug report to English: [paste text]

查阅内容（规格文档与术语表）

如果你连接了规格文档或填写了术语表，可以向 BugIt 询问有关你项目的问题。

YOU TYPE (EXAMPLES)

What is [your term]?

Walk me through the checkout flow

What's new in specs?

重复检测

这在每次提交前都会自动发生：BugIt 会在你的 tracker 中搜索匹配的工单（即使措辞略有不同）并提醒你，让你永远不会重复提交同一个缺陷。

强力功能（如果你已开启）

输入这个	它的作用
Triage digest	即时站会摘要：按区域/严重程度分类的未解决缺陷、最旧的标记项。
Export bug to file	将报告保存为 Markdown/CSV/JSON 而不提交（在 tracker 尚未设置好之前非常好用）。
Undo last filing	在你的 tracker 允许的情况下撤销最近一次的工单/评论提交。
（自动）	紧急缺陷的 SLA 标记 · 分类专属字段。

完整命令列表

```
YOU TYPE
help
```

.....随时显示 agent 内的每一条命令。

随时安全练习
输入 `dry run`，BugIt 会撰写整份报告但**不会**提交任何内容，非常适合学习或测试。

第三部分 · 打造专属设置：添加你项目的知识

开箱即用时，BugIt 是一个白板式的 QA agent。以下是购买后教它了解你项目的方法。不需要写代码，大多只是在聊天中和它对话。你添加的一切都只保存在你的电脑上。

1 · 你团队的用语（术语表）

这能让翻译和类别推测对你的项目更加准确。

- 1. 在文件列表中，打开 `.github/glossary/terms.template.md`。
- 2. 在表格中填入你团队的用语：功能名称、界面名称、条目名称、缩写，以及（如果你使用两种语言）它们的翻译。
- 3. 保存文件（`Ctrl+S`）。就这样，BugIt 从此会使用这些确切的术语。

快捷方式
在 `Begin setup` 期间打开"术语表自动填充"，BugIt 会和你已有的工单中建议术语，你只需逐一确认。

2 · 你的缺陷风格（团队风格）

想让工单完全匹配你团队的格式（标题风格、严重程度标签、必填字段）？你不需要描述它，只需**展示**给它看：

- 1. 在聊天中说："match my team's style."
- 2. 粘贴你 tracker 中一张现有工单的截图。
- 3. BugIt 会研究它（在本地进行，并先剥离任何个人数据），保存你的格式，然后在之后每一份工单上都遵循它。

3 · 你的类别、平台与字段

只需用大白话告诉 BugIt，它就会替你更新设置：

YOU TYPE (EXAMPLES)

```
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 · 你自己的工具（自定义连接）

使用的工具不在内置列表中？像连接内置工具一样连接它：

YOU TYPE

```
Add a custom MCP server
```

给它起一个简短的名称并提供地址（必须是 `https://`）；BugIt 会替你连接并进行健康检查。

5 · 随手纠正即可

最简单的方法：当 BugIt 起草工单时，直接修正你不喜欢的地方，比如一个标签、措辞或严重程度。开启“从你的修改中学习”（推荐）后，它会记住你的偏好并在下次应用。你的项目知识会自然而然地积累起来，而且没有一项会离开你的电脑。

你添加的一切都属于你，且只保存在本地

你的术语表、团队风格、类别和学习到的偏好都保存在你的电脑上，在每次更新前都会备份，绝不会发送给任何人。

6 · 与团队共享你的设置（团队许可证）

一旦你按自己的想法设置好了术语表、团队风格和 tracker 选择，就可以用一条命令把完全相同的设置给团队其他成员，而不必在每台机器上重复步骤 1-3：

YOU RUN (ONCE, AFTER YOUR SETUP IS THE WAY YOU LIKE IT)

```
python tools/share.py export
```

这会生成 `config.share.zip`，把你的 tracker/崩溃工具/通讯选择、术语表和团队风格打包在一起，不含任何密码或令牌。把它发给你的团队。

EACH TEAMMATE RUNS

```
python tools/share.py import config.share.zip
```

他们会立即获得和你完全相同的术语、缺陷风格和 tracker 设置，不用重新输入类别，也不用为团队风格重新粘贴截图。他们自己的许可证和设备设置不受影响。

如果导入更改了数据发送的位置

如果共享的设置新增了一个自定义连接或通知地址，BugIt 会先准确展示发生了什么变化，而不是悄悄应用它。它也会先为该队友当前的设置生成快照，因此 `Restore my settings` 可以在导入结果不符合预期时撤销此次导入。

第四部分 · 更新、备份与安全

获取更新（一条命令）

有新版本可用时，BugIt 会在你输入 `hi` 时提及一次。只要你的许可证有效，v1.x 的免费更新即包含在内。安装方法：

YOU TYPE

Update

1. BugIt 会先为你的文件做一次全新备份。
2. 它会下载新版本，并检查其真实性与完整性（更新经过加密签名，伪造或被篡改的更新会被拒绝）。
3. 它会在不改动你的设置、术语表、团队风格、许可证或令牌的情况下完成安装。
4. 它会要求你重新加载 VS Code（`Ctrl+Shift+P` → 输入 "Reload Window" → 回车）。开始一个新的聊天，你就用上了新版本。

永远不需要删除文件夹

与首次安装不同，更新是原地进行的。你永远不需要删除或重新下载任何东西，你的设置也始终会被保留并备份。

哪些可以放心手动编辑，哪些不行

安全，且每次更新都会保留： `config.json`、`.github/instructions/house-style.instructions.md`（见第三部分，这是自定义行为的官方支持方式）、`.github/glossary/terms.template.md`、`.github/instructions/learned.instructions.md`，以及 `.vscode/mcp.json`。

不安全：请勿手动编辑这些文件

agent 文件（`.github/agents/bugit-qa-agent.agent.md`）、`.github/instructions/` 下的任何其他文件，以及 `tools/` 中的所有内容，都是产品本身，而非你的设置。更新会直接静默覆盖它们，而且和上面的文件不同，没有备份可以恢复它们。BugIt 会在启动时以及安装更新前再次检查这一点，如果发现改动会提醒你，但最保险的做法就是干脆不要编辑它们。

备份与恢复

你输入	它的作用
<code>Back up my settings</code>	保存你的配置、术语表、团队风格、MCP 端点和学习数据的本地快照；你签名的设备权益和凭据值不包含在内。
<code>List backups</code>	显示每一份已保存的快照及其日期。
<code>Restore my settings</code>	回滚到某个快照（并先为你当前的状态生成快照，所以恢复操作本身也是可撤销的）。

每次更新前自动进行

BugIt 总是会在安装新版本前先备份，因此更新绝不会丢失你的设置。如果发现任何异常，`Restore my settings` 即可修正。

安全：受保护的内容

✅ 未经你同意，不做任何事

每条工单和评论都会预览；不可逆的操作需要你输入 `FILE IT`；仅输入“是”不会执行。

🔧 Dry-run = 零写入

说 `dry run`，BugIt 就只读取，绝不提交、评论或通知。干运行会阻止 BugIt 内置的 Python 助手写入，并指示代理拒绝对跟踪器的写入；该拒绝是遵循 BugIt 指令，并非平台级锁定；评估时请使用只读凭据。

🔑 文件中不含密钥

令牌保存在你操作系统的保险库中，绝不以明文文件保存，也绝不发送给我们。

🔒 运行在你自己的电脑上

它只与你连接的工具和你自己的 AI 模型通信。

你的数据与隐私

BugIt 发送给 Taskivator 的唯一内容是许可证/更新数据：你在浏览器中、在 BugIt Portal 上完成的 BugIt 账户登录，一个单向哈希的设备 ID，以及你为这台设备批准的 Solo 或 Team 权益。这里没有任何激活码；你的密码始终留在浏览器中。你的缺陷报告、规格文档、术语表、设置和令牌绝不会离开你的电脑。没有遥测，没有分析，绝不会有。完整详情见你 BugIt 文件夹中的 `PRIVACY.md` 文件。

风险与重要注意事项：请务必阅读

AI 可能出错

BugIt 使用 AI 模型起草报告，它可能误读细节或凭空捏造内容。在你确认（输入 `FILE IT`）之前，请务必阅读预览内容。你批准的才是最终提交的内容。

它会提交到你真实的 tracker

一旦你确认，就会创建一个真实的工单。想练习？用 `dry run` 撰写报告而不提交。

你项目的安全由你负责

你如何使用 BugIt，以及你项目、数据和 tracker 的安全，都是你自己的责任。这些保障措施能降低风险，但不能替代你的审阅。

BugIt 目前做不到的事

经过测试的字段映射（自动设置 severity/priority）为 Jira + Azure DevOps（Azure DevOps 通过 Microsoft 的远程 MCP 公开预览接入）；Confluence 使用同一套引导式 Atlassian 连接；Sentry、GitHub、Linear 和 Notion 为实验性并会在你的账户上实时验证；其余一切都通过你自己的兼容 MCP server 连接（由你设置，BugIt 会协助）；它使用你自己的 AI（Copilot 或你自己的 OpenAI/Anthropic 密钥），而非内置模型；脱敏是尽力而为；且仅在 VS Code 中运行。一个许可证 = 一台设备（Solo）或最多 5 名成员（Team）。效果和一致性也取决于 Copilot 中回答你的是哪个 AI 模型：更强的模型比非常轻量/免费的模型更可靠地遵循安全步骤。

第五部分 · 故障排查与常见问题

几乎每个小问题都能在几秒钟内解决，而且最快的解决办法通常就是直接问一下助手。

🌟 先试试这个：让 AI 帮你修复

无论出现什么问题（无论是在设置期间还是日常使用中），你最快的解决办法几乎总是**直接在聊天中间助手**。用大白话描述发生了什么（例如 "the setup stopped halfway" 或 "my tracker bridge won't connect"），然后让它帮你修复。AI 能够当场诊断并修复大多数问题。在给我们发邮件之前，请先试试这个办法，它能在几秒钟内解决绝大多数问题。

应用 AI 的修复：那个 "Keep" 按钮

如果助手通过修改文件来修复问题，VS Code 会显示一个小的 Keep / Undo 提示条。如果是你要求它修复的，并且你对结果满意，点击 **Keep**，这个改动只会应用到你自己电脑上的这份副本。点击 **Undo** 则丢弃它。你 Keep 下来的内容不会与任何人分享。

你看到的	该怎么做
"Activate to start"	输入 <code>Activate</code> ；BugIt Portal 会在你的浏览器中打开。登录，选择你的 Solo 或 Team 权益，批准这台设备。还没有账户？到商店购买一个。
浏览器没有打开，或激活没有完成	再次输入 <code>Activate</code> 以重新打开 BugIt Portal，然后完成登录并批准这台设备。激活完全在浏览器中进行，无需粘贴任何内容。
"No tracker enabled"	输入 <code>Begin setup</code> 并选择你的 tracker。
你的 bridge 显示红色 X / 无法连接	打开 <code>.vscode/mcp.json</code> ，点击 server 上方的 Start。还是卡住？输入 <code>fix servers</code> 并按提示操作。
它一直要求你登录	输入 <code>fix servers</code> ，重启服务器并在浏览器中完成登录。BugIt 从不显示已保存的凭据值。（第一次遇到？见步骤 8c。）
它无法提交缺陷	输入 <code>Check readiness</code> ，它会准确列出缺少的内容（通常是 bridge 未启动或你尚未登录）。
更新后看起来有问题	输入 <code>Restore my settings</code> 进行回滚。
agent 看起来"偏离脚本"	输入 <code>Read and follow all your given instructions</code> ，或开始一个新的聊天。这种情况在轻量/免费的 AI 模型上更容易出现，在 Copilot 的模型选择器中换一个更强的模型会更稳定。
回答感觉很笼统/不针对你的项目	提供更多上下文，或在 <code>Begin setup</code> 期间给它看一张现有工单，让它学习你的风格。重新运行 <code>Begin setup</code> 以进一步优化。
你想更改某项设置选择	输入 <code>Begin setup</code> 并说明要改什么（例如 "add a tracker" 或 "start over"），它只会重新打开对应部分。如果设置只是中途被打断，单独输入 <code>Begin setup</code> 会从中断处继续，而不是重新开始。

内置健康检查

`Check status`（我的工具都连接好了吗？）· `Check readiness`（我还差什么才能提交？）。想要更深入的检查，打开 Terminal → New Terminal 并运行 `python tools/selftest.py`。

常见问题

我需要懂编程吗？

不需要。如果你会安装应用、会打字造句，你就能使用 BugIt。技术部分它都替你完成了。

它会在我不知情的情况下提交缺陷吗？

绝不会。每条工单和评论都会先预览，并且不可逆的操作需要你输入 `FILE IT`；仅输入“是”不会执行。用 `dry run` 可以零写入地练习。

我每次都要启动 `bridge` 吗？

是的。重新打开 VS Code 时，打开 `.vscode/mcp.json` 并点击 Start。你保存的登录信息会被记住；只有 `bridge` 需要重启。

我可以在两台电脑上使用它吗？

Solo 版一次只能用一台（Team 版最多 5 名成员，每位成员一台）。更换到新设备没问题：激活新设备后，旧设备将无法立即续期。其名额要等到旧设备重新联网（确认已交出访问权限），或其当前的 72 小时离线许可证到期后才会释放，因此没有固定的等待时间。输入 `License status` 即可查看。

如果许可证服务器宕机了怎么办？

你可以继续工作。成功完成在线激活后，缓存的许可证可让此设备离线工作最多 72 小时，之后需要在线验证。因此在这段时间内，一次服务中断，或只是没有网络的周末，都不会打断你工作。

我需要 GitHub Copilot 吗？

这是最简单的方式。如果你没有它，可以在终端中运行 `python tools/setup_wizard.py` 完成设置，BugIt 也可以搭配你自己的 OpenAI/Anthropic 密钥独立运行（`python standalone/run.py`）。

我的数据是私密的吗？

是的。BugIt 发送给 Taskivator 的唯一内容是许可证/更新数据：你在浏览器中、在 BugIt Portal 上完成的 BugIt 账户登录，一个单向哈希的设备 ID，以及你为这台设备批准的 Solo 或 Team 权益。这里没有任何激活码；你的密码始终留在浏览器中。绝无遥测。你的工单本身只会发送给你连接的 AI 模型和 tracker，绝不会发给我们。

我可以编辑 BugIt 的文件吗？

你应该自定义属于你的部分：术语表、团队风格和设置（第三部分）。只是不要禁用授权/激活功能。如果你在工具本身发现了真正的缺陷，请发邮件联系支持，以便在下次更新中为所有人修复它。

它支持哪些缺陷跟踪系统？

Jira 和 Azure DevOps 提供引导式设置和经过测试的字段映射（自动设置 severity/priority），Jira 通过 Atlassian Rovo MCP，Azure DevOps 通过 Microsoft 的远程 MCP 公开预览。Sentry、GitHub、Linear 和 Notion 使用 BugIt 设置并在你的账户上实时验证的已知端点（在这些检查通过之前为实验性）；GitLab、Shortcut、YouTrack、Bugzilla、ClickUp、Asana 和 Trello 通过你自己的兼容 MCP server 连接。BugIt 撰写工单，并通过你启用的任意连接器进行提交。在设置时选择你的 tracker，之后随时可用 `Begin setup` 更换。

提交前它会发现重复缺陷吗？

会。在写入任何内容之前，它会在你的 tracker 中搜索相似的报告（即使措辞略有不同），并把匹配项展示给你，让你避免重复提交同一个缺陷。是否继续仍由你决定。

它能用不止一种语言撰写工单吗？

可以。在设置时添加第二语言，之后每份报告都会用两种语言撰写，分成独立的段落，并使用你的术语表确保产品术语准确无误。它绝不会随意发挥翻译。

我可以附加截图吗？

直接把图片粘贴到聊天中即可。BugIt 会读取它，清除它注意到的任何敏感信息，并在你确认后把它附加到工单上。

它会在我的报告中隐藏密码和令牌吗？

开启脱敏功能后，它会在提交前从每份草稿中清除邮箱、令牌和 IP 地址。你在预览中始终能看到完整的文本。

我不小心提交了一个工单，能撤销吗？

预览和输入 **FILE IT** 的确认步骤能阻止大多数误操作发生在写入之前。如果你开启了审计日志，输入 **Undo last filing** 。否则，像平时一样在你的 tracker 中关闭或编辑该工单即可。

它能帮我验证或关闭一个缺陷吗？

可以。请求一条验证评论，它会针对现有工单起草一条清晰的通过/不通过说明（使用你的语言），你确认后它才会发布。

我可以把它用于不止一个项目吗？

每个 BugIt 文件夹只针对一个项目的 tracker 进行设置。对于另一个项目，要么重新运行 **Begin setup** 指向新的位置，要么为每个项目保留一份独立的副本。

我如何获取更新？

有新版本发布时，BugIt 会在你开始聊天时提及。输入 **update** ，它会原地安装：你的术语表、团队风格和设置都会被保留，并且会先进行备份。

我的许可证到期后会怎样？

期限结束后，请从你的账户续费；你的设备会在下一次在线签入时应用续费后的权益。72 小时的离线租约是另一回事，它只针对许可证服务器的临时中断，而非到期的期限。随时输入 **License status** 查看。

如果我续费或再购买一次，天数会累加吗？

不会。续费会替换你当前的权益；期限重新开始计算，未用完的天数不会结转，所以请在当前期限快结束时再续费。

第六部分 · 许可证与支持

许可条款（通俗摘要）

完整的、具有约束力的条款在你 BugIt 文件夹中的 **LICENSE** 文件里，以该文件为准。简而言之：

主题	通俗说明
授权使用，而非出售	你购买的是使用 BugIt 的许可，而非所有权。Taskivator 保留全部权利。
你的席位	Solo = 同时 1 台设备；Team = 最多 5 名成员。你的账户和权益是你个人专属的，请勿分享、转卖或公开发布。
撤销	被共享、退款、拒付或滥用的访问权限可能被撤销。
可以自定义，但.....	可以自由编辑你的配置、术语表和模板，但不要禁用或绕过授权/激活机制。
你的责任	你如何使用 BugIt，以及你项目的安全，都是你自己的责任。每份报告提交前都由你审阅并批准。这些保障措施是辅助手段，而非保证。
"按现状提供"，责任限制	按现状提供，不附带任何担保。在法律允许的范围内，责任上限为你支付的金额，不包括间接或后果性损失。
期限与适用法律	为期一年的许可证，自你激活当天起算（一次性购买，不自动续订），受日本法律管辖。退款通过你购买的商店办理。
续费不会累加	续费会替换你当前的权益：其有效期从激活时重新开始计算，未用完的天数不会结转，因此请在当前期限快结束时再续费。

你文件夹中的两份文件

`LICENSE`（完整条款）和 `PRIVACY.md`（隐私声明）。激活并使用 BugIt 即表示你接受 `LICENSE`。

获取帮助

请先查看上面的故障排查和常见问题，它们几乎覆盖了所有情况。然后，按以下顺序：

1 · 让 AI 帮你修复 ★

你最好的第一步：在聊天中描述问题，并让助手帮你修复。如果它修改了文件并且看起来没问题，点击 Keep 应用它（仅限你自己的电脑）。

2 · 运行健康检查

`Check status` · `Check readiness` · 或在终端中运行 `python tools/selftest.py`。

3 · 恢复

`Restore my settings` 可撤销一次糟糕的改动或更新。

4 · 联系真人

只有在指南和 AI 都无法帮到你时：访问 bugit.dev 或发邮件至 support@bugit.dev（技术支持仅提供英文）。购买后 7 天内遇到设置问题？我们会帮你把它跑起来，或协助你通过商店办理退款。

请勿在支持邮件中包含机密的项目信息

每个项目都不一样，你的很多工作内容可能是机密的。如果你确实要给我们发邮件，请只用一般性的语言描述 BugIt 的问题，不要粘贴机密代码、缺陷详情、规格文档、截图或来自你项目的数据。对于发送给我们的任何机密信息，Taskivator 概不负责，我们收到的任何机密内容都会被立即删除。请尽可能先让 AI 助手帮忙，它可以直接在你的编辑器中解决大多数问题。

感谢你选择 BugIt。

提交规范的工单，跳过重复项，把时间还给自己，一次一个缺陷。